Using MsPASS for Seismology Data Processing

Intro to HPC Systems

Ian Wang





Supercomputer






DALCO

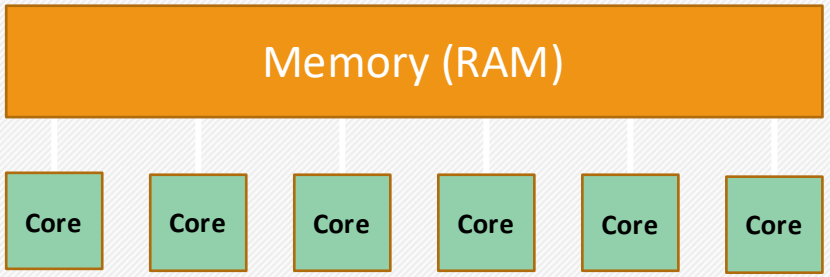


Top500 Ranking

Rank	Site	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	National Supercomputing Centre in Shenzhen (NSCS) China	**LineShine** - LingKun, LX2 304C 1.55GHz, LingQi, Kylin OS	13,789,440	2,198.40	2,735.82	42,220
2	DOE/NNSA/LLNL United States	**El Capitan** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE	11,340,000	1,809.00	2,821.10	29,685
3	DOE/SC/Oak Ridge National Laboratory United States	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE	9,066,176	1,353.00	2,055.72	24,607
	Maybe on your desk	High-end GPU workstation - 24-core CPU, NVIDIA GeForce RTX 5090	24 CPU cores 170 GPU SMs		0.00164	~1.0

Shared Memory

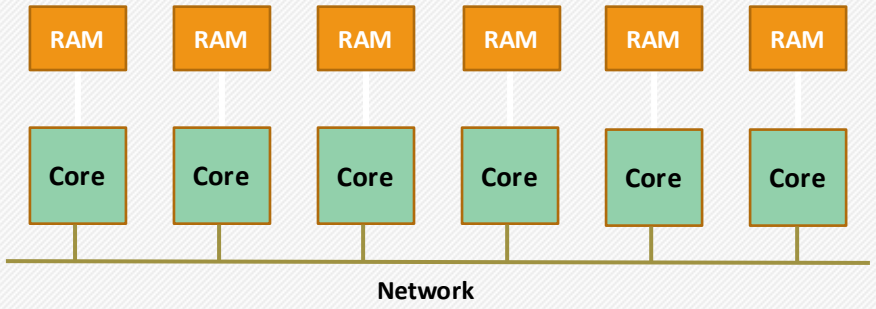
- All cores share a common pool of memory (RAM)
- The programming challenge is coordination: how to avoid competing for access to the same puzzle pieces (memory)
- A **single executable** spawns independent threads and manages threads' access to data





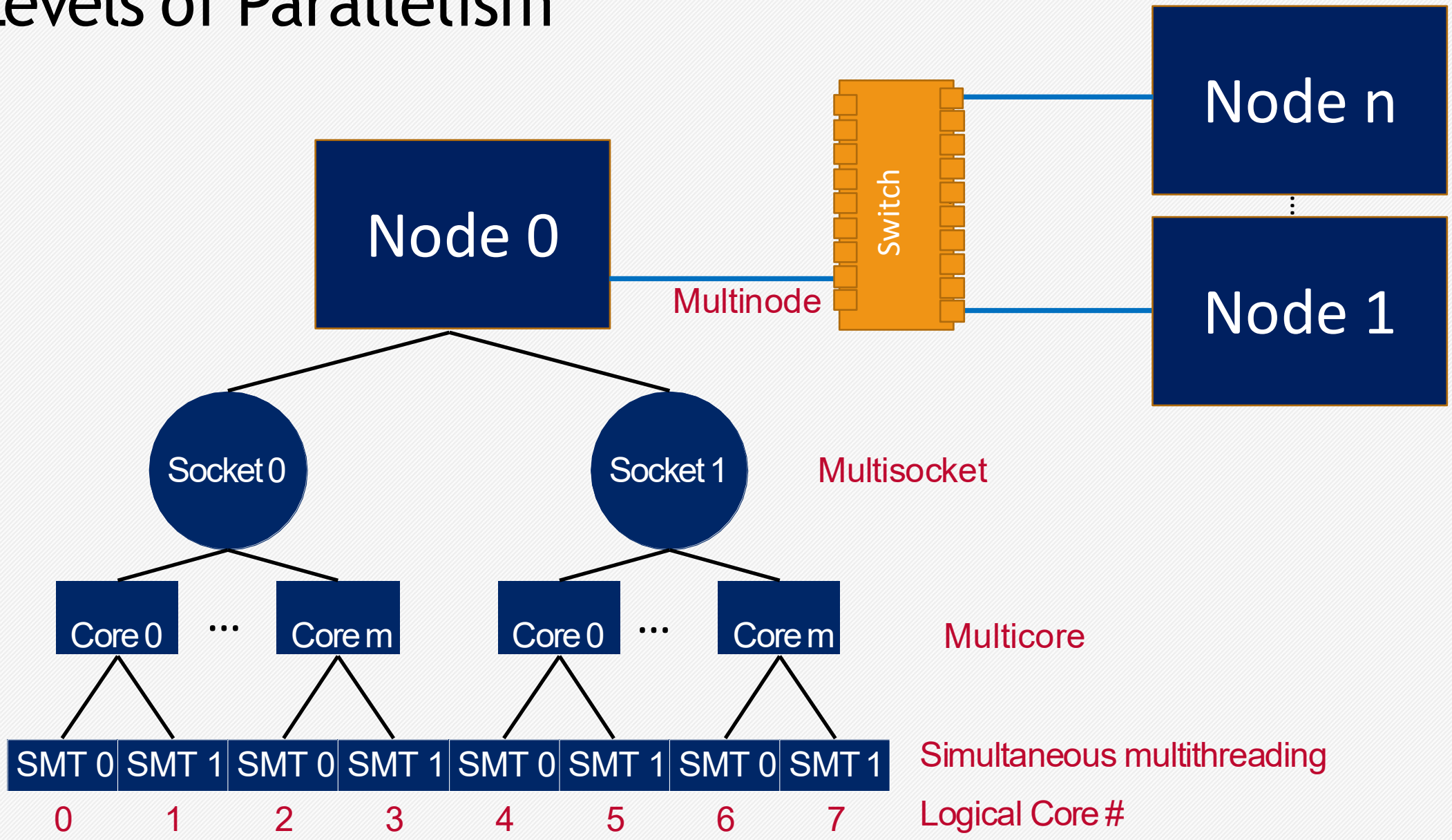
Distributed Memory

- Each core has its own memory (RAM), inaccessible to other cores
- The programming challenge is communication: how to share puzzle pieces (data)
- Every assigned core runs a separate copy of the same executable -- a “rank aware” task





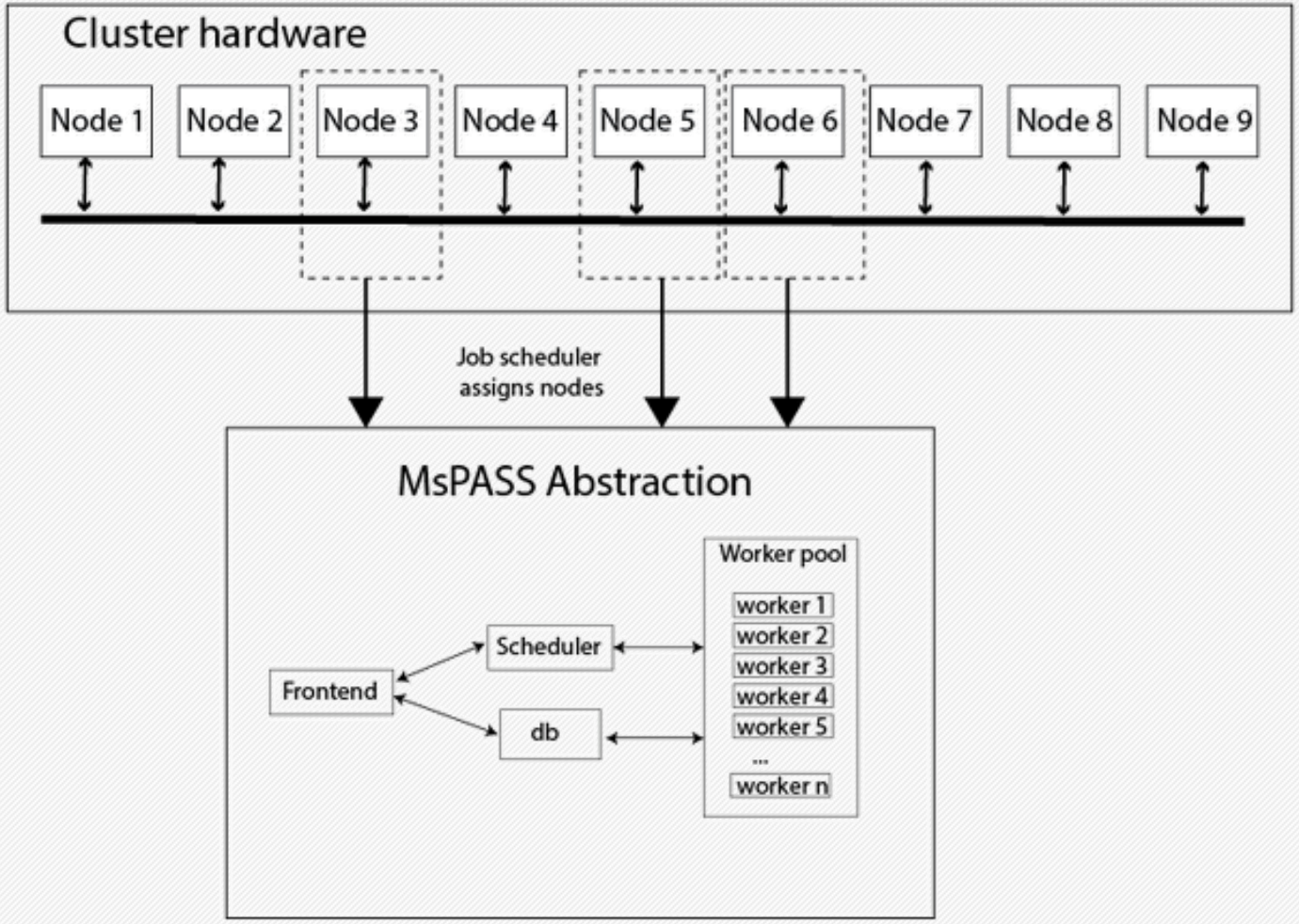
Levels of Parallelism



HPC Configuration of MsPASS

- Options
  - Python package
    - Conda or pip install can work
    - Infamous package incompatibilities worse on giant systems
    - Requires running auxiliary services (MongoDB and dask) that can be problematic
    - Recommended only for development and advanced users
  - Containerized – recommended approach
    - Shell batch setup (legacy – no longer recommended)
    - Python launcher (what we will use today)
- For either option need to understand service model (next topic)

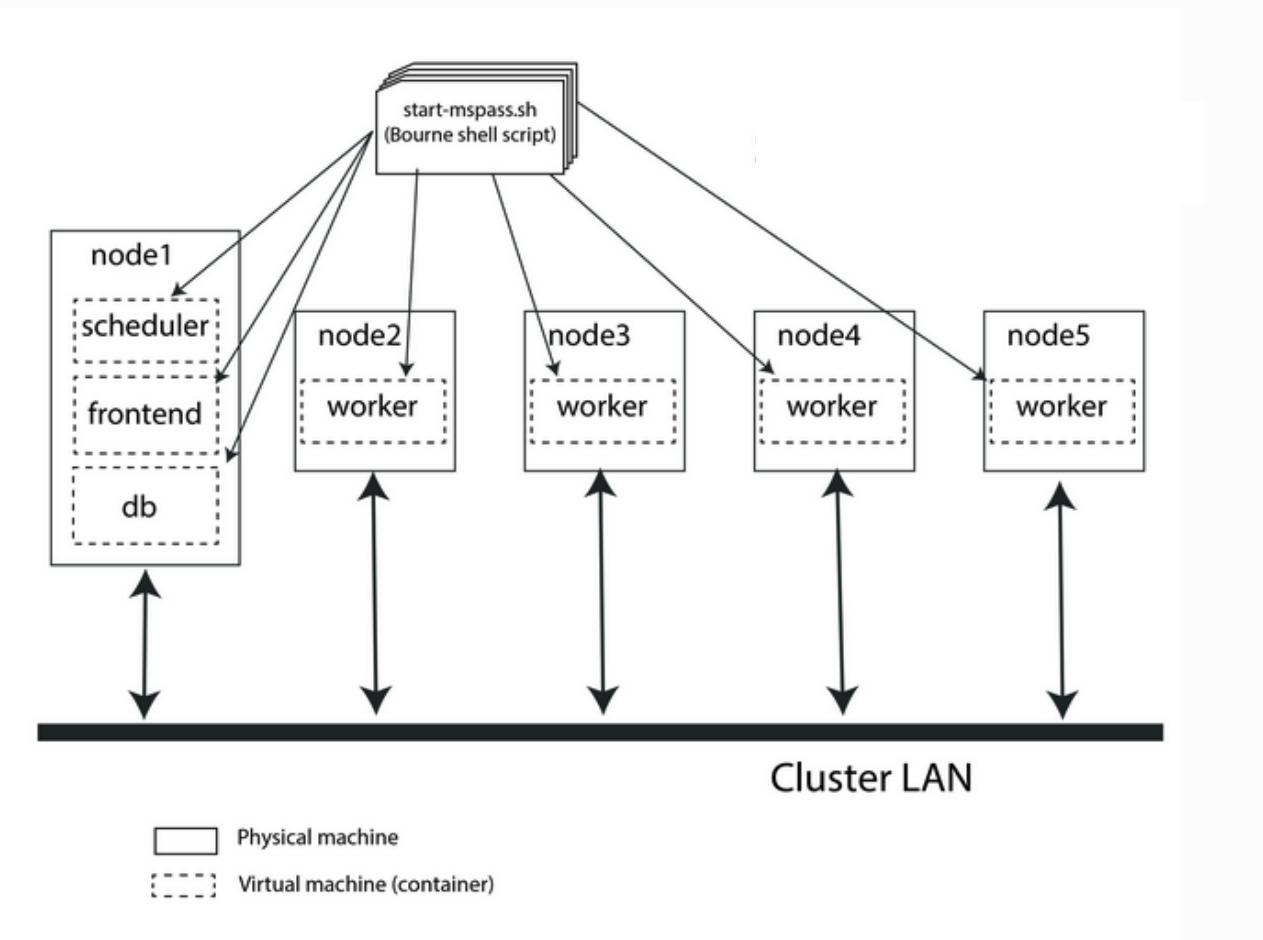
MsPASS Service model and HPC



MsPASS Services

- scheduler
- worker(s)
- db
- frontend

Container example

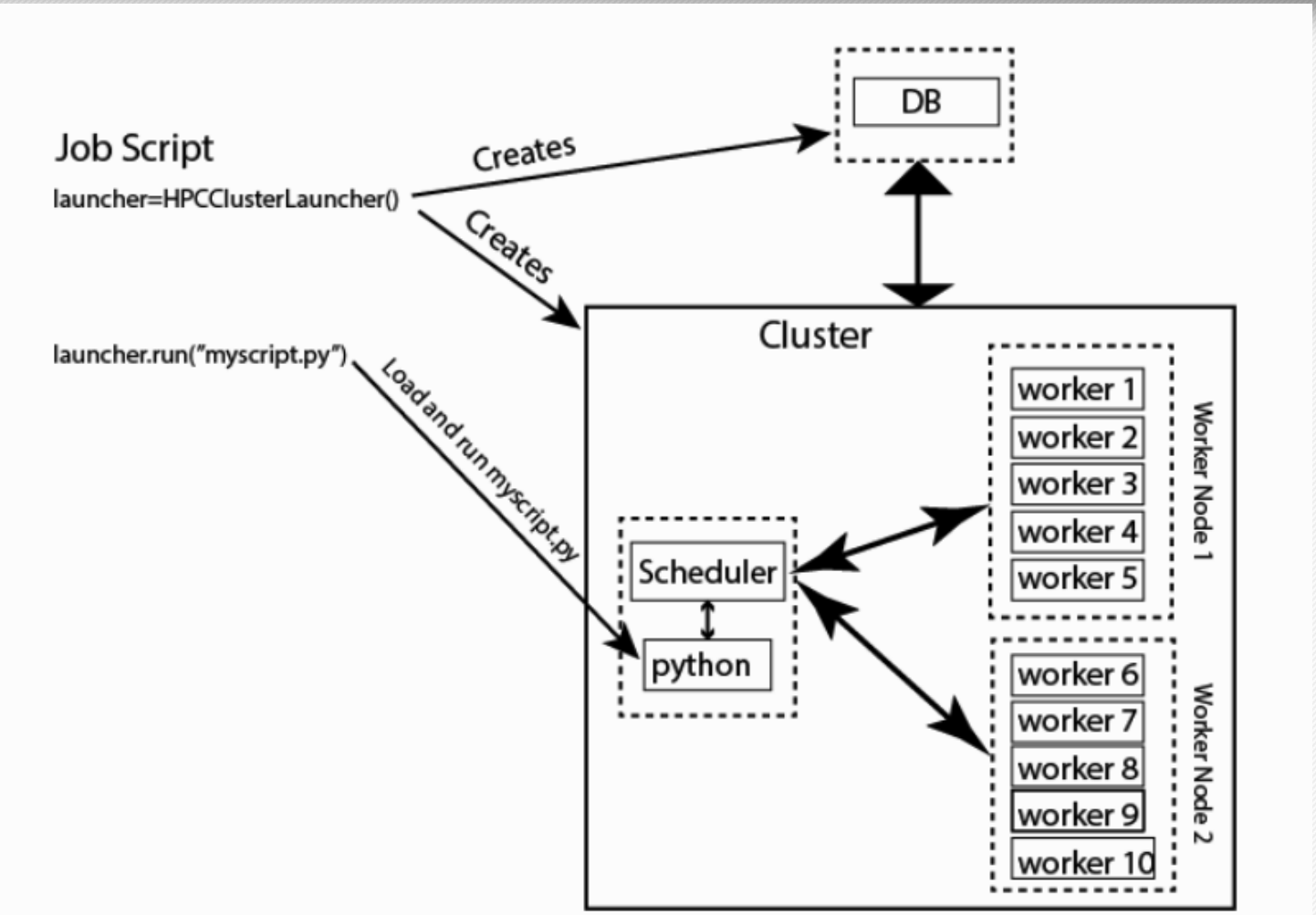


Points about this figure:

- Each dashed box is a service
- Each solid box is a compute node
- Containers are an implementation detail
- start script (top)
  - Run when container launched
  - Same virtual machine runs in different “roles” == service

From MsPASS User's Manual HPC configurations section

HPCClusterLauncher



Measuring Scalability

How fast can we bake a cake?

- Assume (with no claim of realism)…
  - Two hours to prepare the cake for baking
  - Another half hour to bake the cake

- So total time is…
  - **`Prep: 120 minutes`**
  - **`Bake:  30 minutes`**
  - **`Total: 150 minutes`**

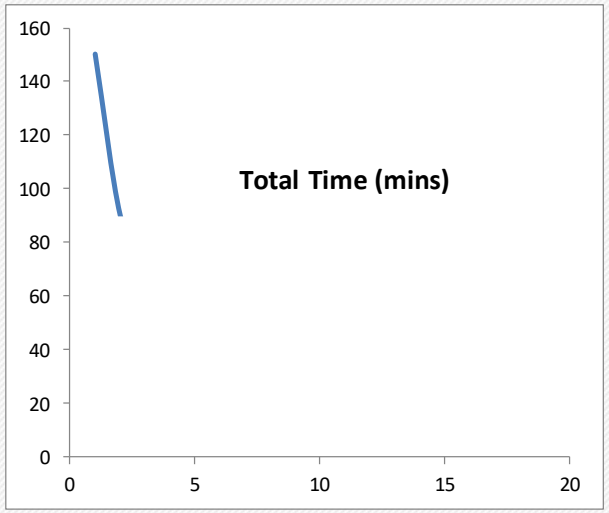
How fast can we bake a cake?

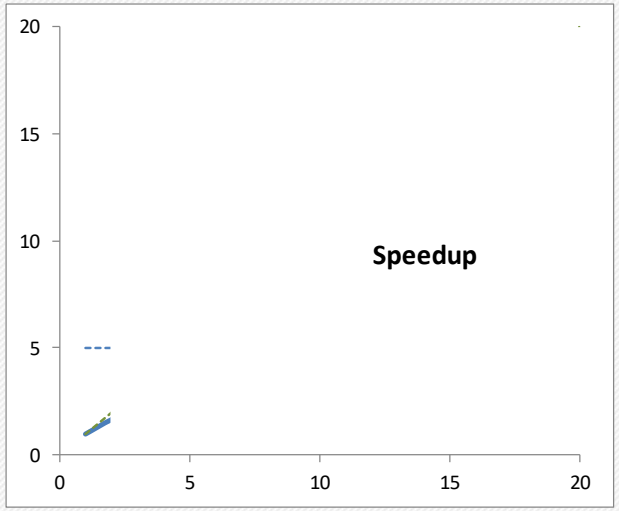
- But what if we have two cooks?
  - Assume they work perfectly together
  - No wasted time, no overhead

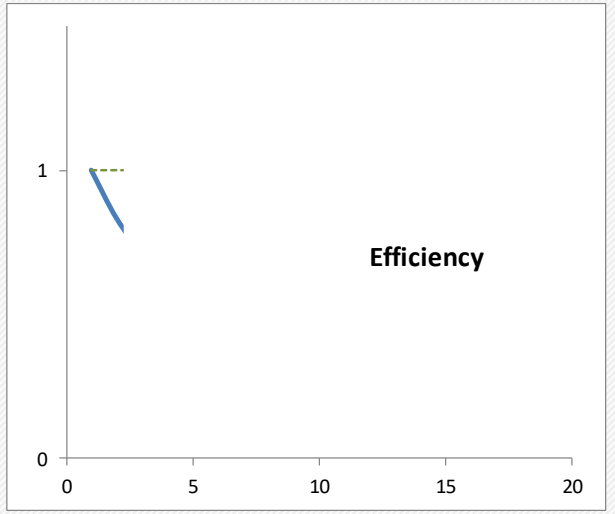
- Then total time is…
  - **Prep: 120/2 = 60 minutes**
  - **Bake: 30 minutes**
  - **Total: 90 minutes**

How fast can we bake a cake?

cooks	prep*	bake	total	speedup	efficiency
1	120/1 = 120.0	30.0	150.0	1.0	1.0
2	120/2 = 60.0	30.0	90.0	1.7	0.8

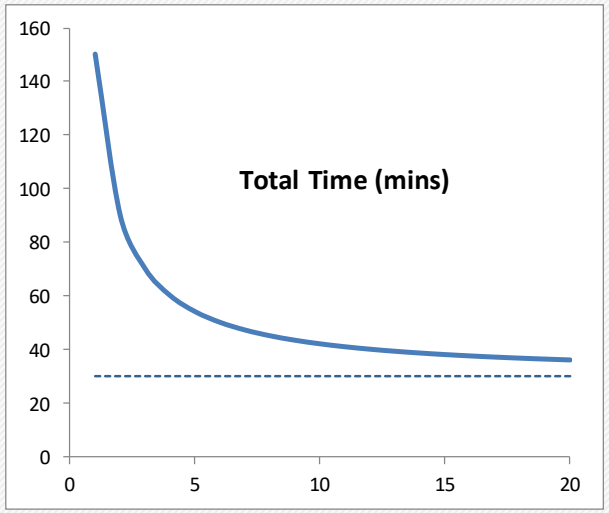


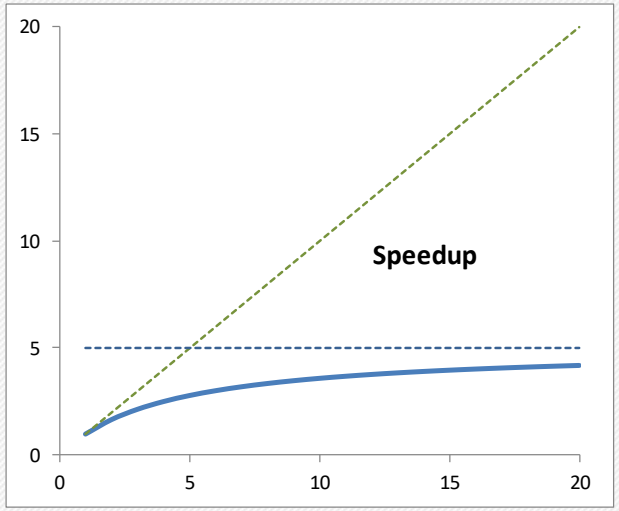


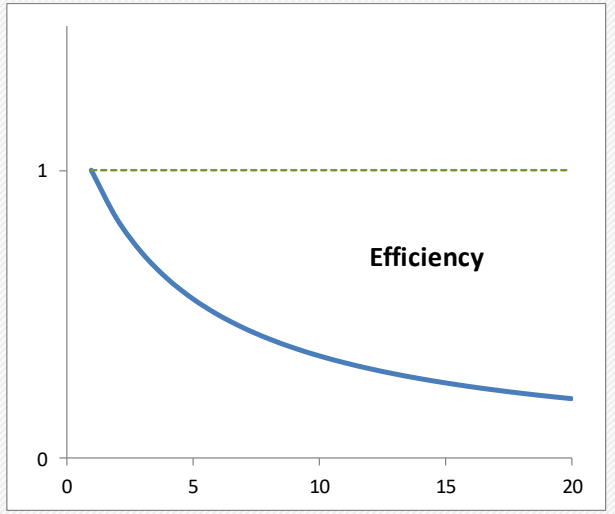


How fast can we bake a cake?

cooks	prep*	bake	total	speedup	efficiency
1	120/1 = 120.0	30.0	150.0	1.0	1.0
2	120/2 = 60.0	30.0	90.0	1.7	0.8
3	120/3 = 40.0	30.0	70.0	2.1	0.7
4	120/4 = 30.0	30.0	60.0	2.5	0.6
5	120/5 = 24.0	30.0	54.0	2.8	0.6
19	120/19 = 6.3	30.0	36.3	4.1	0.2
20	120/20 = 6.0	30.0	36.0	4.2	0.2



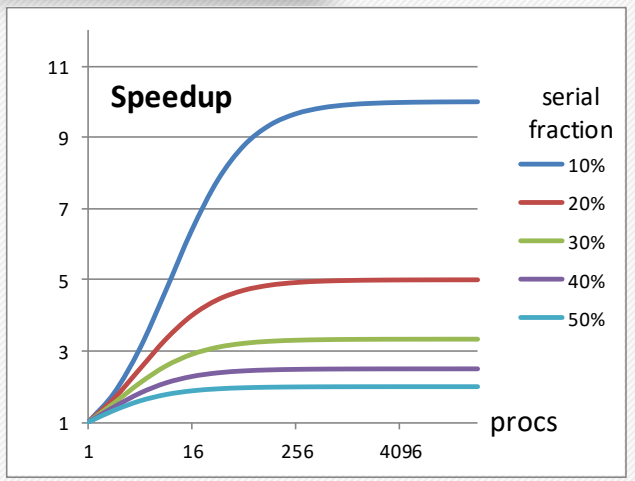


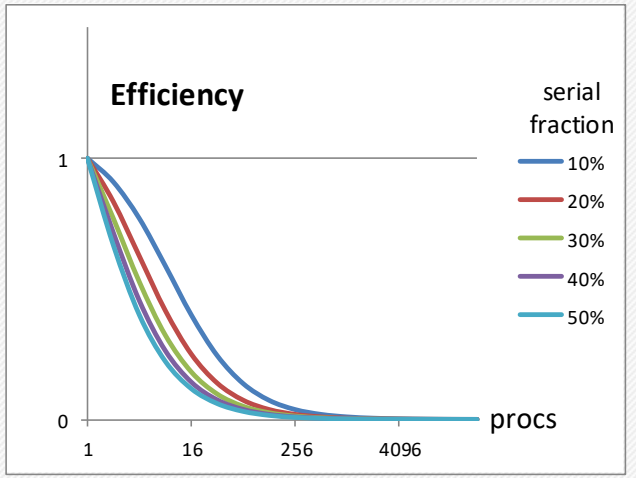


Amdahl's Law: Strong Scaling

- Back-of-napkin scalability bounds

$$S_p = \frac{T_1}{T_p} \leq \frac{t_s + t_p}{t_s + \frac{t_p}{p}} \rightarrow \frac{1}{\alpha}$$





Amdahl's Law: Strong Scaling

- Back-of-napkin scalability bounds

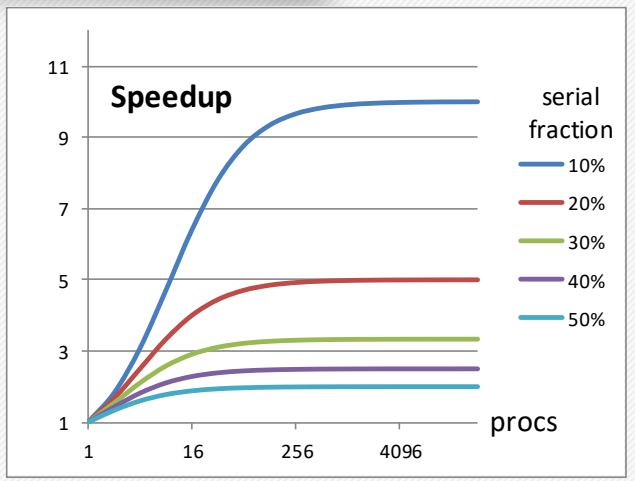
$$S_p = \frac{T_1}{T_p} \leq \frac{t_s + t_p}{t_s + \frac{t_p}{p}} \rightarrow \frac{1}{\alpha}$$

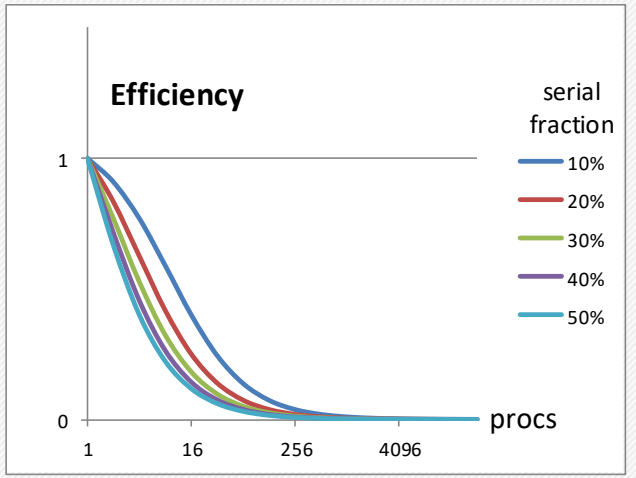
actual speedup

perfect on parallel section

infinitely fast on parallel section

reciprocal of serial fraction



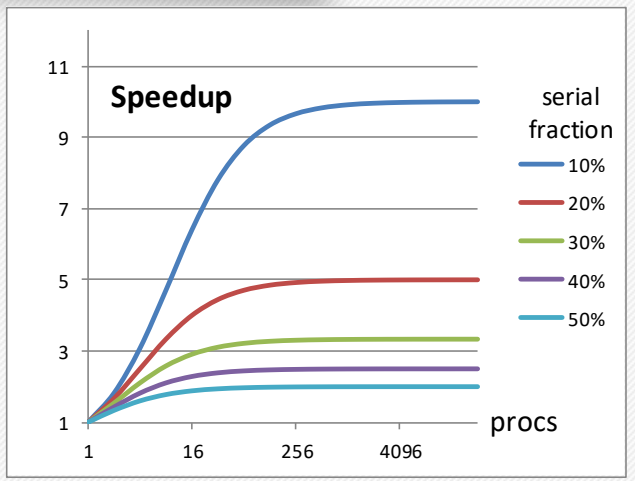


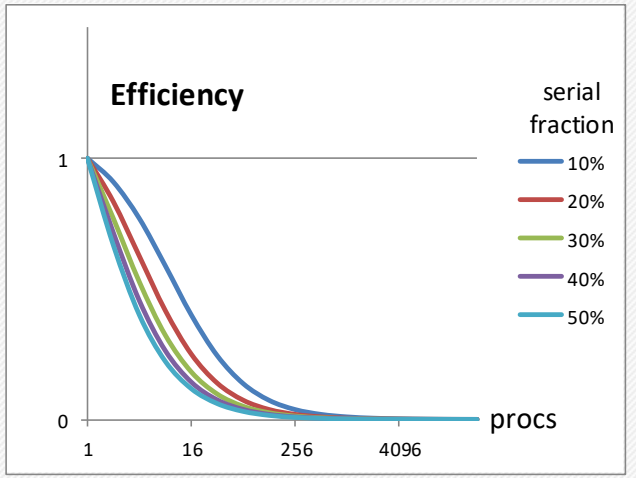
Amdahl's Law: Strong Scaling

- Back-of-napkin scalability bounds

$$S_p = \frac{T_1}{T_p} \leq \frac{t_s + t_p}{t_s + \frac{t_p}{p}} \rightarrow \frac{1}{\alpha}$$

- Under reasonable assumptions…
  - Serial fraction is a severe constraint
  - As you add processors, you reach a point of diminishing returns
  - Can't run faster than serial time
  - Speedup bounded by reciprocal of serial fraction





Thank you!